



## طراحی سامانه‌های میکروسرویس

نیم‌سال دوم ۱۴۰۴-۰۵

استاد: دکتر علی ابریشمی - سیدحسین میرحسینی

پاسخ‌دهنده: معین آعلی - ۴۰۱۱۰۵۵۶۱

تمرین اول

## فهرست مسائل

|   |       |         |
|---|-------|---------|
| ۱ | ..... | مسئله ۱ |
| ۱ | ..... | الف     |
| ۱ | ..... | ب       |
| ۲ | ..... | ج       |
| ۳ | ..... | مسئله ۲ |

## پاسخ مسئله‌ی ۱.

## الف

موجودیت یا Entity نماینده‌ی یک داده‌ی واقعی در سیستم و پایگاه داده است. این کلاس‌ها معمولاً مستقیماً با جدول‌های دیتابیس در ارتباط هستند و دارای ID می‌باشند. همچنین ممکن است شامل منطق و رفتار مرتبط با دامنه‌ی مسئله باشند.

مثال:

```
class User {  
    Long id;  
    String name;  
    String password;  
}
```

DTO یا Data Transfer Object شیئی است که فقط برای انتقال داده بین لایه‌های مختلف سیستم استفاده می‌شود. DTO معمولاً منطق خاصی ندارد و فقط شامل اطلاعات موردنیاز است.

مثال:

```
class CreateUserDto {  
    String name;  
    String password;  
}
```

تفاوت اصلی این است که Entity نماینده‌ی داده‌ی اصلی سیستم و مرتبط با دیتابیس است؛ اما DTO فقط برای انتقال داده و کاهش وابستگی یا افزایش امنیت استفاده می‌شود.

## ب

اصل CCP بیان می‌کند که کلاس‌هایی که به دلیل مشابهی تغییر می‌کنند، بهتر است در یک پکیج یا ماژول قرار بگیرند. به بیان ساده، اگر چند کلاس معمولاً هم‌زمان تغییر می‌کنند، باید کنار یکدیگر باشند تا مدیریت تغییرات ساده‌تر شود.

برای مثال، کلاس‌های مربوط به فاکتور مانند: Invoice, InvoiceService, InvoiceValidator بهتر است در یک پکیج مشترک قرار بگیرند؛ زیرا تغییر قوانین فاکتور احتمالا باعث تغییر همه‌ی آن‌ها می‌شود. مزیت این اصل:

- کاهش اثر تغییرات روی بخش‌های دیگر
- افزایش نگهداری‌پذیری پروژه
- ساده‌تر شدن توسعه و تست

## ج

Decoupling به معنی کاهش وابستگی بین بخش‌های مختلف سیستم است. هرچه اجزای سیستم مستقل‌تر باشند، توسعه و نگهداری ساده‌تر خواهد بود. منظور از سطح Deployment این است که اجزای سیستم چگونه منتشر و اجرا می‌شوند. در یک سیستم کاملاً Decoupled، هر بخش می‌تواند به صورت مستقل Deploy شود. برای مثال در معماری Microservice سرویس کاربران و سرویس پرداخت و سرویس سفارش هرکدام جداگانه اجرا و منتشر می‌شوند. اما در معماری Monolith، معمولاً کل سیستم باید با هم Deploy شود.

مزایای Deployment مستقل:

- سرعت بیشتر در انتشار تغییرات
- کاهش وابستگی بین نسخه‌ها
- امکان توسعه‌ی مستقل توسط تیم‌ها
- کاهش ریسک هنگام انتشار تغییرات

## پاسخ مسئله‌ی ۲.

## الهام‌گیری Architecture Clean از اصل DIP

اصل وارونگی وابستگی شامل دو بند اساسی است:

۱. ماژول‌های سطح بالا نباید به ماژول‌های سطح پایین وابسته باشند؛ هر دو باید به انتزاع وابسته باشند.
  ۲. انتزاع‌ها نباید وابسته به جزئیات باشند، بلکه جزئیات (پایاده‌سازی‌های واقعی) باید وابسته به انتزاع‌ها باشند.
- در Clean Architecture، نرم‌افزار به لایه‌های متمرکزی تقسیم می‌شود که هسته مرکزی آن شامل بالاترین سطح از سیاست‌ها و منطق تجاری برنامه (Domain و Use Cases) است، و لایه‌های بیرونی شامل ابزارها و مکانیزم‌های سطح پایین (Frameworks, Databases, UI) هستند.
- قانون طلایی این معماری: جهت وابستگی‌های کدهای منبع همیشه باید رو به داخل و به سمت سیاست‌های سطح بالا باشد.
- تناقض معماری و حل آن به کمک وارونگی در عمل یک تناقض پیش می‌آید: یک Use Case سطح بالا (مثلاً RegisterUser) برای انجام کار خود نیاز دارد داده‌ای را ذخیره کند یا ایمیلی بفرستد، که این‌ها عملیاتی سطح پایین و مربوط به ابزارهای بیرونی هستند.
- Clean Architecture این تناقض را دقیقاً با الهام از DIP حل می‌کند: به جای اینکه Use Case مستقیماً به کدهای دیتابیس وابسته شود، یک اینترفیس در داخل لایه خود تعریف می‌کند. لایه دیتابیس در بیرون، این اینترفیس را پیاده‌سازی می‌کند. در نتیجه، وابستگی زمان کامپایل کاملاً وارونه شده و به سمت داخل متمایل می‌شود، در حالی که جریان داده در زمان اجرا به سمت بیرون است.

## مشکلات ساختاری که با رعایت این اصل حل می‌شوند

- **Coupling شدید:** بدون DIP، هرگونه تغییر در جزئیات دیتابیس (مثلاً مهاجرت از PostgreSQL به MongoDB) یا تغییر در ابزارهای بیرونی، منطق اصلی کسب‌وکار را خراب می‌کند. DIP هسته سیستم را از نوسانات زیرساخت ایزوله می‌کند.
- **تست‌پذیری ضعیف:** وقتی منطق برنامه به سیستم‌های واقعی متصل باشد، نوشتن Unit Test سریع غیرممکن است. با وابستگی به انتزاع‌ها، می‌توان در محیط تست به راحتی ساختارهای فرضی را تزریق کرد.
- **اختلال در توسعه موازی:** تیم توسعه لایه تجاری می‌تواند بدون منتظر ماندن برای نهایی شدن ساختار دیتابیس یا سرویس‌های شبکه، کدهای خود را به طور مستقل بنویسد و تست کند.

## نحوه متصل شدن دو لایه به یکدیگر در زمان اجرا

بر اساس قانون وابستگی، لایه‌های درونی هیچ دیدی به لایه‌های بیرونی ندارند. اما در زمان اجرا، جریان کنترل برای دسترسی به دیتابیس باید از لایه تجاری خارج شود. برای حل این تناقض بدون مخدوش شدن مرزهای معماری، از الگوی Dependency Injection استفاده می‌شود. در این روش، لایه بالا نیازمندی خود را به صورت یک اینترفیس اعلام می‌کند. سپس در بیرونی‌ترین نقطه برنامه یعنی Composition Root (مانند تابع main)، پیاده‌سازی واقعی دیتابیس ساخته شده و به لایه بالا تزریق می‌شود. مراحل کدی این اتصال به شرح زیر است:

تعریف انتزاع و مدل در لایه سطح بالا (Core)

```

1 package usecases
2
3 // Domain Entity -
4 type User struct {
5     ID    string
6     Name  string
7 }
8
9 // UserRepository -
10 type UserRepository interface {
11     GetByID(id string) (*User, error)
12 }

```

پیاده‌سازی مستقل در لایه سطح پایین (Infrastructure)

```

1 package infrastructure
2
3 import (
4     "database/sql"
5     "errors"
6     "usecases"
7 )
8
9 type DBUser struct {
10     ID          string
11     FullName    string
12     CreatedAt   string
13 }
14
15 type SQLUserRepository struct {
16     db *sql.DB
17 }
18
19 func NewSQLUserRepository(db *sql.DB) *SQLUserRepository {
20     return &SQLUserRepository{db: db}
21 }
22
23 func (r *SQLUserRepository) GetByID(id string) (*usecases.User, error) {
24     query := "SELECT * FROM users WHERE id = $1;"
25
26     var dbUser DBUser
27
28     err := r.db.QueryRow(query, id).Scan(&dbUser.ID, &dbUser.FullName, &dbUser.CreatedAt)
29     if err != nil {
30         if errors.Is(err, sql.ErrNoRows) {
31             return nil, nil
32         }
33         return nil, err
34     }
35
36     domainUser := &usecases.User{
37         ID:    dbUser.ID,
38         Name:  dbUser.FullName,
39     }
40
41     return domainUser, nil
42 }

```

تزریق وابستگی در نقطه ورود (Composition Root)

```
1 package main
2
3 import (
4     "database/sql"
5     _ "github.com/lib/pq"
6     "infrastructure"
7     "usecases"
8 )
9
10 func main() {
11     dbConn, err := sql.Open("postgres", "postgres://user:pass@localhost/db")
12     if err != nil {
13         panic(err)
14     }
15     defer dbConn.Close()
16
17     userRepoImpl := infrastructure.NewSQLUserRepository(dbConn)
18
19     userUseCase := usecases.NewUserUseCase(userRepoImpl)
20
21     user, err := userUseCase.Execute("123")
22 }
```